

GPS Integration Tutorial

Integration flow:

1. Vendor will provide:
 - a. Base URLs for the endpoints from which GolfGenius will send/request data, such as [vendor.com/api/...](#)
 - b. A vendor token which will be sent in every request in order to confirm the validity of the request. Will be sent in the body, can be sent in the header as well if needed.
 - c. A site ID for a test facility which will be used for tests and implementation
2. GolfGenius will create a production and staging account for the vendor, adding the people involved from the vendor as customer managers
 - a. GPS Integration will be enabled and the Site ID provided earlier will be connected to the account
3. Vendor will implement the APIs which GolfGenius will retrieve the data from
 - a. [vendor.com/api/carts/{site_id}](#) - this api will respond with a list of carts available at the facility with the provided Site ID, and their status, for example:

None

```
[ {  
  "car_number"=>77,  
  "is_connected"=>true,  
  "is_on_course"=>true,  
  "is_staged"=>false  
},  
{  
  "car_number"=>223,  
  "is_connected"=>false,  
  "is_on_course"=>false,  
  "is_staged"=>false  
},  
...  
]
```

This will be called whenever the manager visits the Golf Genius page responsible for creating cart assignments

- b. [vendor.com/api/courses/{site_id}](#) - this api will respond with a list of courses available at the facility with the provided Site ID, and their information, for example:

```
None
[ {
  "name" => "Golf Course Name",
  "course_id" => vendor_course_id,
  "hole" => "eighteen",
},
{
  "name" => "Golf Course Name Front 9",
  "course_id" => vendor_course_id,
  "hole" => "front9",
},
{
  "name" => "Golf Course Name Back 9",
  "course_id" => vendor_course_id,
  "hole" => "back9",
},
...
]
```

This will be called when editing courses in order to map courses in Golf Genius to courses in Vendors system

4. At this point, cart assignments can be performed in Golf Genius:

The screenshot shows the 'Players' and 'Carts' sections of the Golf Genius interface. The 'Players' section has a table with player names and a grid of player cards. The 'Carts' section shows a list of carts with their names and locations. The 'Assign' button is highlighted in orange.

Players			
First5 Last5	Garry Potter	Preston Osborne	
Frank Lyons	Lawrence Leonard	Billy Garcia	Javier Thomas
Joe Tribiani	Mona Matthews	Opal Farmer	Tiffany Newman
First1 Last1	First3 Last3	Rose Roberts	Mable Brown
Golfer4 NoMailHCP	Golfer1 Full	Rudy Santos	Shelly Greene
First4 Last4	Jon Rowe	Lester Dunn	Bradley Boyd
Golfer5 OnlyLast...	Player One	Alan Barnett	Rudy Bishop

Carts ☐ Show Connected Carts ☐ Show Staged Carts

- 1 Lakes (front 9) / Orchard (front 9)
- 224 Lakes (front 9) / Orchard (front 9)
- 617983 Lakes (front 9) / Orchard (front 9)
- 619166 Lakes (front 9) / Orchard (front 9)
- 803751 Lakes (front 9) / Orchard (front 9)
- 842161 Lakes (front 9) / Orchard (front 9)
- 9840001 Lakes (front 9) / Orchard (front 9)
- 141700764 Lakes (front 9) / Orchard (front 9)

so the Vendor will need to implement an endpoint which will accept a POST request from Golf Genius with the following format. This request will be sent

```
None
{
  vendor_site_id: 50,
  vendor_token: VENDOR_TOKEN,
  carts:
```

```
[{
  vendor_course_id: 124,
  number: 77, (cart number received from Vendor)
  team_id: 12, (if both players are part of same team we send a unique
team id; if this tournament is not played by teams, or if players from
multiple teams in same cart, we send -1),
  starting_hole: "2",
  starting_time: "15:00PM",
  players:
  [{
    player_id: x,
    player_name: y,
    post_scores: [true, true, ... ,true]
  },
  {
    ...
  }]
},
{
  ...
}]
}
```

This endpoint will be used when any changes are made to cart assignments and saved into Golf Genius. Subsequent requests will only send updates.

- starting_hole can have values such as: 1, 2A, 4B, 15A, 18
- post_scores is an array of 18 values which can have 9 "true" and 9 "false" / 9 "false" and 9 "true" representing 18 holes, front 9 and back 9,

5. The vendor will implement the logic to build the scores data and send it to Golf Genius whenever they have updates regarding scores. The endpoint to be used is www.golfgenius.com/api/vendor/scores, and the body should be formed like this:

None

```
{
  "vendor_site_id": 123,
  "vendor_token": VENDOR_TOKEN,
  "scores":
  [{
    "player_id": 87263745,
    "scores": [4, 5, 4, 2, 3, 4, 3, -1, -1, nil, nil, 0, 0, 0, 0, 0, 0,
0]
  }]
}
```

```
{
  ...
}
]
```

“scores” will be an array of 18 elements which will be interpreted in the following way:

- Nil or 0 -> hole not played
- -1 -> leave score unchanged (will keep whatever value it has in Golf Genius)
- 1, 2, 3, ... 20 -> valid score

In case of teams, scores will be sent for all the players and Golf Genius will handle the rest. The response from Golf Genius will be “ACK”. If no response is received, we are not 100% sure that the API request was successful, so we recommend resending the scores.

“Vendor” in “vendor_token” and “vendor_site_id” will be replaced with the vendor name.

6. The vendor will need to implement an endpoint where Golf Genius sends data whenever one of these 3 events occur:
 - a. Saving cart assignments in a round - *will cause the response to include the carts data*
 - b. Setting up/ updating a tournament that is going to be sent to the Vendor
 - c. Generating a new leaderboard for a Vendor tournament

The data will be similar to the payload described at 4th point, but will also have a tournaments array:

None

```
{
  vendor_site_id: 50,
  vendor_token: VENDOR_TOKEN,
  carts:
  [{
    vendor_course_id: 124,
    number: 77, (cart number received from Vendor)
    team_id: 12, (if both players are part of same team we send a unique
team id; if this tournament is not played by teams, or if players from
multiple teams in same cart, we send -1),
    starting_hole: "2",
    starting_time: "15:00PM",
    players:
    [{
      player_id: x,
      player_name: y,
```

```
        post_scores: [true, true, ... ,true]
      },
      {
        ...
      }
    ]
  },
  {
    ...
  }
]
tournaments:
[
  {
    id: 11252,
    name: "Fall Scramble - Fall Scramble division",
    adjusted: false,
    creation_date: "2024-10-14T16:44:27Z",
    date: 2025-02-04T01:00:00Z
    status: ["Not Started", "In Progress", "Completed"], # One of 3
    options
    format: "Stroke", (if no tournament defined yet in our system we
    return "")
    html: HTML code of tournament,
    size: (of html)
  },
  {
    ...
  }
]
```

The endpoint for this API can be the same with the one where Golf Genius sends only cart assignments, or can be different, based on the Vendors preferences.

- Date format is ISO 8601
- Adjusted represents the finalized leaderboard

API endpoints to be implemented by vendor:

1. vendor.com/api/carts/{site_id}
 - a. GET - returns carts
2. vendor.com/api/courses/{site_id}
 - a. GET - returns courses
3. URL TBD by vendor
 - a. POST - accepts carts assignments
4. URL TBD by vendor, can be same with the one at 3
 - a. POST - accepts carts assignments **and tournaments**

Golf Genius API endpoints to be used by vendor:

1. www.golfgenius.com/api/vendor/scores
 - a. An example of the format and encoding details are available at point 5 in the Tutorial
 - b. NOTE: “vendor” is a place holder to be replaced with the same thing as the “vendor” in the rest of the endpoints, such as “yoursite.com/api/courses”, “www.golfgenius.com/api/yoursite/scores”

Sample HTMLs:

Sample HTML code for a simple tournament:

<http://www.golfleaguegenius.com/v2tournaments/23404>

Sample HTML code for a medium-complexity tournament:

<http://www.golfleaguegenius.com/v2tournaments/23407>

Sample HTML code for an advanced tournament:

<http://www.golfleaguegenius.com/v2tournaments/102801>

Glossary (to be updated as needed)

- Token - a agreed upon random string which will be sent in the payload
- Site ID - can be the vendor's internal id for each location/facility, which can have one or multiple courses on the same site.